

خاتمة خوارزميات نظري 4 -

Heap

1. في خاتمة سابقة دقلنا أن أحد الطرق التي نعرف فيها tree هي على شكل مصفوفة أحادية لكن سلبات هذه الطريقة هي أنها تجزئ مساحات فارغة كثيرة

tree ← مصفوفة أحادية

أما فكرة Heap هي بالعكس مصفوفة أحادية (المصفوفة الأحادية حابجي فيها فراغات) يتعامل معها على أنها Binary tree و برسمها كأنها complet tree يعني يعني كل المستويات من اليسار إلى اليمين

1. Heap العادية ما ستفاد منها شيء هي فقط طريقة أخرى لرسم أو تخيل tree لذلك عرفنا نوعين

↪ **Max Heap** هي Binary tree كل عقدة هي أكبر من

ابنائها ← وبالتالي أكبر من أخطائها

↪ **Min Heap** هي Binary tree كل عقدة هي أصغر من ابنائها

← وبالتالي هي أصغر من أخطائها

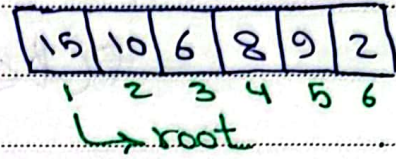
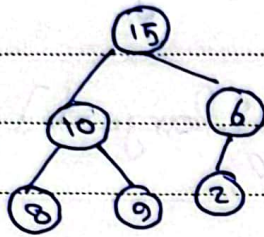
ملاحظة

Max Heap ليست بالمرتبة ان تكون مصفوفة Sorted

هي مصفوفة مرتبة جزئياً أو يوجد علاقة ترتيب

بين عناصرها

أما إذا كان لدينا مصفوفة Sorted فهي Heap



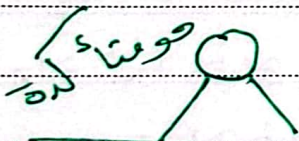
ملاحظة

* الـ Heap هي Binary tree وليست Binary Search tree

* في أعلى الـ Heap هي Sort و تتحول الـ Max Heap
أي أعلى Sort كامل، راح يكلف تعقيد زيادة وانا بيلزميني بس
Sort جدي لأعلى الـ Max heap

سؤال:

لو عندي Subtree معين وبار هي الـ Max heap: مستند الـ root
ما يعرف إذا حتمه الـ Max heap أو لا



لازم أعلى اصلاح للقيمة إذا لزم الأمر

دع الـ MaxHeapify



هي أعلى حويل الـ Heap

Max Heap

الكود:

```

void MaxHeapify (int [ ] A, int i) {
    int r = right (i);
    int l = left (i);
    int largest = i;
  
```



```

if (r ≤ A.length() && A[largest] < A[r])
    largest = r;

```

```

if (l ≤ A.length() && A[largest] < A[l])
    largest = l;

```

```

* if (largest != i) {

```

```

    ← Swap (A, i, largest);

```

```

    MaxHeapify (A, largest);

```

```

}

```

```

}

```

```

int left (int i) {

```

```

    return 2i

```

```

}

```

```

int right (int i) {

```

```

    return 2i+1;

```

```

}

```

```

int parent (int i) {

```

```

    return  $\frac{i}{2}$ ;

```

```

}

```

مبدأ الوقوف موجود حيناً لأنو بـ سطر *
 اذا خفت سطر == فراح سطر عي حالو
 السابح

السؤال الأساسي أنو أنا في array عشوائية
بدي أعلاها Max heap بدون ما أعلى sort
كامل عليها راج ~~هو~~ استخدم التابع Max heapify
كيف دابا

مثال

0 1 5 15 2 4 1 7

ملاحظات

1. الأوراق ما في داي أعلى عليها استدعاء (زيادة بلا طمعة)
2. بيلش استدعاء من آخر array لأول
3. عليه Max heapify بتصححها إذا كانت subtree
واليمين وال subtree ليسار هي Max heap
4. أما بهي الحالة أنا ما عرف شو عندي بال array و ما في
أعلى استدعاء على root و حق إذا علت استدعاء
على root راج راج swap بين ما يصححها
5. و الطريقة هي تحويل array لحالة انو subtrees
اليمين واليسار هما بطرين جديين راج استدعاء على root
ياخذ كل فرع على حدا و يصلح ابتداء و يرجع يصلحو لهيك
بيلش من آخر الصفوفة
6. راج ليش استدعاء من 7 ما في ابتداء بلك 1 و 4 نفس لشي
لهيك الاستدعاء عن الأوراق عالو طمعة
هلق عند 12 عندها ابن واحد يات هو 7 وهو أكبر منها
ف راج swap بينهم

5	15	7	4	1	2
---	----	---	---	---	---

نرجع نعمل عند 15 ضابطة أكبر من أبناءها 1 و 4
 نعمل نعمل استعداء عند 5 نعمل عندها استعداء أبناءها
 15 و 7 نعمل swap بين الأكبر 15 والابن 5

15	5	7	4	1	2
----	---	---	---	---	---

هذه هي array لا بين هي Max heap وهو Sorted

الهدف

التحقق

لأنها complete فارجح ان يكون لها نصير خفية في ما في
 مكانات خفية في ارتفاعها $\log$ اذا كانت ال هي
 ال root الاسود هو $\log$ من الاستعدادات تكلفة
 ال استعداد الواحد $O(1)$ تم استعداد n استعداد في التحقق
 $n \log(n)$ وهي فعلة كبيرة فارجح ان وصلنا
 اصلاً لأن ال استعداد عند الأوراق هو $\log$ هو $O(1)$
 والاحسن اننا ما نعمل استعداد اصلاً عند الأوراق

في ملاحظة

الأوراق هو يأخذ مستوى حصة الأوراق يأخذ ال array
 ادبي عدد

بما ان عدد العقدة الداخلية بعيد الأوراق
 عدد الأوراق هي أقل بواحد من الداخلية

↑

نظام ~~الترتيب~~ ~~الترتيب~~
 في اننا نعمل استبعاد من نصف array مكان ما اعلم استبعاد
 على الأوراق فمن $\frac{n}{2} \log n$ وقت ما نوصلو

for (int i = A.length/2 -> 1)

MaxHeapify(A, i)

طريق شجرة لائحة من هاد كلو ١١٥

اللائحة الأولى

Heap Sort هو نوع من أنواع Sort يتحقق $n \log n$

بعد باستخدام Heap

مثال

0	15	5	7	1	4	6
---	----	---	---	---	---	---

مكان ما من

index = 1

مع array عشوائية بدي العمل

Sort اول خطوة جعلها Max heap بناءً على Max heap

هو $n \log n$ * هوون بالمثل هي بالفعل Max heap

+ اننا نأخذ اقل Sort على نصف array يعني نأخذ اقل

على مكان ثاني

الفترة من ذلك اننا نأخذ root أكبر قيمة فعلية

MaxHeapify نعيد جعل swap بين root و آخر عنصر

مرجع سبدي مرة ثانية بعد ما صغر ال length واحد

ونعك swap

هون بيصير أكبر عنصر قاعد بجاو، واضح

0	6	5	7	1	4	15
---	---	---	---	---	---	----

نعمل استثناء بدون 15 ونعمل

Maxheapify

0	7	5	6	1	4	15
---	---	---	---	---	---	----

(أخترني Heap من array)
يعني بدون 15

رجع سنيل root ونجربو آخر سنيل أوبير نفس

العمليات لأدخل ل Sorted array

0	1	4	5	6	7	15
---	---	---	---	---	---	----

~~Maxheapify~~ ~~Array~~ ~~Array~~ ~~Array~~

في الحقيقة بأسوأ الأحوال
بناء الماكس هيب + $n \log n$ وهي تكلفة Maxheapify
و n متغيرة في كل مرة

المادة، الثانية

Priority queue، تلك الأولوية

هو عبارة عن queue مرتب حسب الأهمية

أي أن كل عنصر في queue لديه أولوية

والعنصر صاحب الأولوية الأعلى يتم معالجة أولاً

نغض النظر عن ترتيب الزمنية (طابور وإسطوانات)

ن

الإضافة فيه يتم في المكان المناسب أما الوصول فهو
 حراً للعنصر ذو الأولوية الأعلى، الحذف يتم من الجذر
 root أيضاً

كيف يتم تحقيق هذه البنية؟

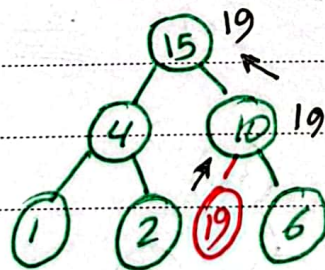
يمكن تحقيقه باستخدام sorted array بسبب لتكلفة
 في array عند الإضافة يكون من تعقيد $O(n)$ بسبب
 الإزاحة فنعتمد طريقة مكلفة

الطريقة الأمثل لتحقيق هذه البنية بتعقيد $O(\log n)$ هي باستخدام
 Heap أول شيء يعرف queue ويعرف عليها إجراءات
 ① top (بشيء أول عنصر بدون ما أتجوز) ② enqueue
 ③ dequeue

II آلية الإضافة

نقوم بإضافة العنصر الجذر في نهاية المصفوفة ونقوم
 بـ maxheapify عليه لرفع العنصر إلى مكانه
 الصحيح إذا كان مكانه في الأعلى يسمى هذه العملية
 بـ Heapify-up فهي التأكيد من أن العنصر الجذر الذي
 أضفناه يصعد إلى مكانه الصحيح حسب الأولوية

مثال



نريد إضافة 19

نصفه في آخر

المصفوفة ثم نقوم

بـ maxheapify إلا أن نتوسع 19 في مكانه الصحيح

عملية الحذف

تعلننا سابقاً أن عملية الحذف غير موجودة في ال Heap فقط نقوم بعملية swap بين العنصر ونقله حجم المصفوفة فيتم حذف العنصر

عندما نريد حذف عنصر من priority queue نأخذ العنصر صاحب

الأكبر أولوية (Root) ونقوم بعملية swap بينه وبين

آخر عنصر في المصفوفة ونقله حجم المصفوفة فيتم

حذفه لكن عندما قمنا بعملية swap ابقنا كبرائو

احصل شرط Maxheap فنقوم بعملية Maxheapify من ال Root

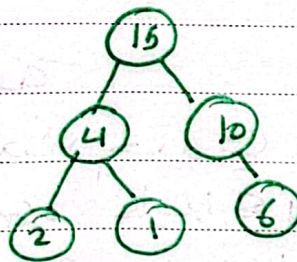
ويسمى هذه العملية Heapify-down لكن أن يعود لترتيب

الصحيح

مثال

أول شيء نلاحظه

بين 15 و 6

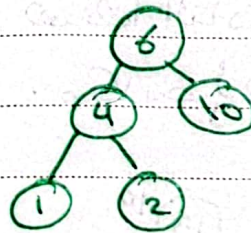


قمنا بتغيير المصفوفة

فقمنا بحذف 15

لكن اقل شرط

Maxheap



قمنا بعملية

Heapify-down

لأنه نحتاج لترتيب

الصحيح للHeap

